

LLM Training Syllabus — v2

Revised from the perspective of someone who has run frontier post-training programs.

What was wrong with v1

Five structural criticisms, in order of severity:

1. **It was a topic list, not a research program.** The meta-skill isn't knowing what GRPO is. It's knowing how to run an experiment that tells you something true, cheaply, and how to know when a small-scale result will transfer. That's now Module 0.
2. **Zero coverage of behavior, character, and safety training.** This is roughly half of what separates a frontier model from a competent fine-tune, and — for a model going into Indian government offices — it's a procurement requirement, not a nice-to-have. Three new modules.
3. **Synthetic data and self-improvement flywheels were missing entirely.** This is where most modern capability gain actually comes from. Serious omission.
4. **Reward modeling got one bullet.** At frontier scale, RM quality is frequently *the* bottleneck — better RMs beat better RL algorithms.
5. **Evaluation was Module 13.** It should be near the front. You build the measuring instrument before the thing you're measuring.

Also missing: multilingual transfer (for a project whose entire premise is Indic), long-context training as a discipline, tool use as a *training target* rather than an eval target, and model merging/checkpoint selection.

Legend: [core] · [skim] · [you already have this]

PART I — RESEARCH METHODOLOGY

Module 0 — Experimental discipline

The module that actually separates labs that ship from labs that burn compute. Nothing below this line matters if you get this wrong.

- **[core] Small-scale proxies and transfer.** You will run 95% of your experiments at 1B or below. The skill is knowing which conclusions transfer to 35B and which don't. Rule of thumb: *relative* orderings of data mixtures and objectives usually transfer; *absolute* hyperparameters usually don't; anything involving emergent capability doesn't transfer at all. Establish this empirically for your own setup with a three-point scaling ladder (e.g. 0.5B / 1.5B / 4B) before trusting any small-scale result.
- **[core] One variable at a time.** Obvious, universally violated. The moment you change the data mix and the LR and the loss masking together, you have learned nothing and spent money. Keep a change log per run.
- **[core] Fitting your own scaling laws** as a prediction tool, not as literature. Run the ladder, fit the curve, predict the big run's loss, then check. If your prediction is off, your ladder is broken and you find out for \$200 instead of \$20,000.
- **[core] Compute budgeting across a research program.** A frontier team spends more compute on ablations than on the final run. Decide your split up front — I'd suggest 60% ablations / 25% main run / 15% reserve for the run you'll have to redo — and track against it.
- **[core] Seeds and variance.** Every headline result needs ≥ 3 seeds. A 2-point gain inside seed variance is not a result. Most published deltas in this field would not survive this test.
- **[core] Negative results are results.** Keep a written record of what didn't work and why. You will otherwise re-run it in four months.

Do: before any real training, build the experiment tracker, the change log, and the scaling ladder. This is a week of work that pays for itself the first time you'd otherwise have chased a phantom improvement.

Module 1 — Mental model of a training step

Be able to narrate what happens between `loss.backward()` and the next forward pass in memory-layout terms. You know the inference side; training adds retained activations, optimizer state, and gradient reduction.

- **[core]** Memory accounting per parameter. AdamW mixed precision: 2 bytes (bf16 weight)
 - $4 \text{ (fp32 master)} + 4 \text{ (exp_avg)} + 4 \text{ (exp_avg_sq)} = 14 \text{ bytes/param}$ before gradients or activations. Derive it for a 35B model and check against real OOMs.
- **[core]** Activation memory vs parameter memory — and why sequence length hurts more than parameter count in long-context agentic training.
- **[core]** Precision: why bf16 not fp16, what fp8 training keeps in high precision, where fp4 is and isn't real.
- **[core]** Where NaNs come from: attention logit overflow, near-zero division in normalization, bad masking producing all-`-inf` rows, corrupted optimizer state after a bad resume.

Do: write a training loop for a ~50M model in plain PyTorch. No Trainer, no Accelerate.

PART II — OPTIMIZATION

Module 2 — Optimization mechanics

- **[core]** SGD → momentum → Adam → AdamW, specifically why decoupled weight decay matters.
- **[core]** Gradient clipping by global norm. Know what constant clipping is *hiding*.
- **[core]** Weight decay's effect on the landscape; why it's disabled on norms and biases.
- **[skim]** Optimizer state sharding (ZeRO-1) as the cheapest memory win.

Modern optimizers

- **Orthogonalized matrix updates:** Muon (SGD+momentum, then Newton–Schulz iteration to orthogonalize the update), Scion, Gluon, PolarGrad.
- **Structured preconditioners:** Shampoo (Kronecker-factored gradient statistics), SOAP (Shampoo stabilized with Adam), Kron, SPlus, DyKAF.

Key results: SOAP and Muon consistently outperform AdamW at multi-billion scale and hold up at batch sizes to 100M tokens where AdamW degrades. They’re mathematically related — turning off the EMA in Shampoo reduces its update to exactly Muon’s.

AdamW remains easiest to scale because its state is elementwise. **Optimizer–architecture co-design is emerging:** in Mamba-hybrids, Muon on dense linear layers + AdamW on the 1D conv layers worked best.

Papers: *Muon is Scalable for LLM Training* · *SOAP* (2409.11321) · *SOAP, Muon, and Beyond* (2607.20548, with the Emerging-Optimizers codebase).

Calibration: for post-training a 35B on domain data, AdamW is fine. Learn this to read papers critically — to spot when a claimed gain is optimizer-driven, not method-driven.

Module 3 — Learning rate, schedules, hyperparameter transfer

Single most common source of wasted GPU-hours.

- **[core]** Warmup: what it protects against.
- **[core]** **WSD is the modern default.** Warmup 0.5–2%, stable plateau 80–90%, decay 10–20%. Cosine requires committing to a total step count up front, making every intermediate checkpoint suboptimal and continual training awkward. WSD lets you decay at any point, resume from stable-phase checkpoints, and — underrated — **change the data mixture during decay** to upweight high-quality data.
- **[core]** Fine-tuning schedules are a different animal: peak LR drops 10–100x, warmup shrinks or disappears, cosine-to-zero over 1–3 epochs, constant LR fine for LoRA.
- **[core]** **Never copy a peak LR from a paper at a different scale.** Recompute from μP transfer or a scaling-law estimate. Mid-training loss spikes almost always trace to peak LR too high, warmup too short, or an LR fine at small batch and not at production batch.
- **[skim]** WSD-S for continual learning. Power Scheduler. Batch-size scheduling.

PART III — DATA

Module 4 — Data engineering

- **[core]** Deduplication: exact, near-duplicate, MinHash/LSH.
- **[core]** Decontamination against your eval sets — before you have results.
- **[core]** Chat templates and loss masking. The most common silent SFT bug is computing loss over prompt tokens.
- **[core]** Tokenizer mechanics: BPE, vocabulary construction, fertility, and what breaks when you swap or extend one — embedding reinit, tied weights, special token IDs.
- **[core]** Packing vs padding; cross-document attention masking.
- **[skim]** Quality classifiers, perplexity filtering, curriculum ordering.

Module 5 — Human data operations (*NEW*)

The part nobody writes papers about and every frontier lab spends enormous effort on. If you're building Indian back-office environments, you are running an annotation program whether you plan to or not.

- **[core]** Writing an annotation guideline that survives contact with annotators. Ambiguity in the guideline shows up as noise in your reward model, which shows up as reward hacking, which you'll misdiagnose as an RL problem.
- **[core]** Inter-annotator agreement (Cohen's/Fleiss' kappa). Below ~ 0.6 your task is underspecified, not your annotators.
- **[core]** Gold-standard seeding and annotator quality scoring. Rotate hidden gold items through the queue continuously.
- **[core]** Preference data collection design: pairwise vs rating scales, how to avoid position and length bias in the interface itself.

- **[skim]** Expert vs crowd annotation and where the crossover is. For GST filing correctness, crowd labels are worthless — you need domain experts, which changes cost by an order of magnitude and should be in your budget from day one.

Module 6 — Synthetic data and self-improvement flywheels (*NEW — major omission in v1*)

This is where most modern capability gain comes from. It was the single biggest gap.

- **[core] Rejection sampling / best-of-N distillation.** Generate N candidates, filter by a verifier or reward model, SFT on the survivors. Simplest possible flywheel, often most of the gain, far cheaper and more stable than RL. Try this *before* you reach for RL.
- **[core] STaR and self-taught reasoning:** generate rationales, keep the ones that reach the correct answer, retrain, repeat. Directly applicable to verifiable back-office workflows.
- **[core] Constitutional AI as a synthetic-data method.** Historically the earliest large-scale use of synthetic data for RLHF. Two mechanisms: (1) the model critiques and revises its own outputs against written principles, then is fine-tuned on the revisions; (2) the model generates pairwise preference data by judging which completion better satisfies a sampled principle — hence RLAIIF. The reported result was a Pareto improvement: more helpful *and* more harmless, with zero human harmfulness labels. Read the CAI paper as a **data generation technique**, not only as a safety technique.
- **[core] Iteration count and collapse.** Each round amplifies the filter’s biases. Know your grounding signal and how many rounds it survives.
- **[core] Verifier design — the filter is the flywheel.** A weak verifier produces a confidently wrong model.
- **[skim]** Instruction backtranslation; self-instruct; evol-instruct.

Why this matters for you specifically: your compute is unconstrained and your labeled data is scarce. Synthetic data flywheels are exactly the technique that converts the resource you have into the resource you don’t.

PART IV — SYSTEMS AND ARCHITECTURE

Module 7 — Distributed training

[you already have this — mostly]. New part is the training-specific communication pattern.

- Parallelism axes: data, tensor, pipeline, sequence, context, expert.
- ZeRO 1/2/3 and FSDP; activation checkpointing; gradient accumulation and its interaction with normalization and logging.
- Communication/compute overlap; MFU as the efficiency metric.
- **MoE-specific:** expert parallelism, auxiliary load-balancing loss, router z-loss, capacity factor, token dropping, and why MoE training is less stable than dense.

Stacks: Megatron-LM, DeepSpeed, TorchTitan.

Module 8 — Architecture choices that affect trainability

- Pre-LN vs post-LN; RMSNorm; **QK-norm** as the standard fix for attention logit blowup.
- Initialization and depth-scaled init.
- RoPE and context extension (YaRN, position interpolation).
- MHA / MQA / GQA / MLA.
- MoE routing: top-k, shared experts, granularity. Fine-grained MoE gives up to 40x compute-efficiency gains when tuned, but 4–8 expert configs need 2.5–3.5x more training resource than 16–32.
- Multi-token prediction heads — what MTP does to the *training objective*, not just to speculative decoding.

Module 9 — Long-context training (*NEW*)

v1 treated this as one RoPE bullet. It's a discipline, and it's load-bearing for agentic work where a 25-step rollout blows past any short-context budget.

- **[core]** Context extension as a training stage: short-to-long curriculum, how many tokens you actually need at each length, and why naively training at max length

wastes most of your compute.

- **[core] Long-context data is the hard part.** Needle-in-a-haystack is a smoke test, not a training target. You need documents with genuine long-range dependency — which is rare and mostly has to be synthesized.
- **[core]** Attention sinks, and why the first few tokens behave differently.
- **[core]** Position extrapolation vs interpolation; where RoPE base frequency scaling breaks.
- **[core]** Context degradation in agentic loops: after ~25–30 tool calls even 200K-token windows show coherence problems — models re-execute completed steps and lose track of what’s done. Mitigations are architectural (memory, summarization-as-action) as much as they are context-length.

PART V — THE TRAINING STAGES

Module 10 — Pretraining and continued pretraining

- Scaling laws: Kaplan → Chinchilla → the inference-optimal correction. Densing Law.
- **Continued pretraining:** replay ratios against catastrophic forgetting, tokenizer extension, embedding reinit for new script coverage.

Module 11 — Multilingual and cross-lingual transfer (*NEW — and it’s your actual project*)

v1 somehow omitted this from a syllabus for an Indic model. Correcting.

- **[core]** Cross-lingual transfer: how much target-language data you need before the model’s English reasoning transfers to Hindi. Usually far less than intuition suggests — reasoning transfers, knowledge and idiom don’t.
- **[core]** **Tokenizer fertility across scripts.** Devanagari encodes more phonemic content per token than Latin; a tokenizer tuned on English silently taxes every Hindi request in both cost and effective context. Measure fertility per language before you choose a base model.

- **[core]** Code-switching and script-mixing. Hinglish in Latin script is a *different distribution* from Hindi in Devanagari, and your users produce both. Train on both.
- **[core]** The curse of multilinguality: adding languages past a threshold degrades per-language quality at fixed capacity. Decide your language count deliberately.
- **[core]** Evaluation in the target language — translated benchmarks measure translation quality as much as capability. You need natively-authored evals.

Module 12 — Supervised fine-tuning

- **[core]** Loss masking on completions only. Chat template must match inference exactly.
- **[core]** **The over-SFT trap.** OOD generalization peaks early and degrades as SFT continues, even while in-domain keeps improving. A subsequent RL stage restores most of it — up to 99% on Qwen-2.5-7B, 85% on Llama-3.2-11B — but only up to a threshold, past which RL can no longer fully heal the forgetting. Hold out an OOD set and stop on *its* curve.
- **[core]** PEFT: LoRA, QLoRA, DoRA. Rank selection, target modules, merge-back.

Module 13 — Reward modeling (*NEW — expanded from one bullet*)

At frontier scale this is often the binding constraint. A better reward model beats a better RL algorithm almost every time.

- **[core]** Bradley–Terry formulation and what it assumes about transitivity.
- **[core]** **Reward model overfitting and drift.** The RM is trained on the base policy’s distribution; RL immediately moves the policy off that distribution and the RM becomes progressively less valid. This is the mechanism behind most reward hacking. Mitigations: iterative RM retraining on fresh on-policy samples, RM ensembles, uncertainty penalties.
- **[core]** RM calibration — a well-ranked but badly-calibrated RM produces unstable advantages.
- **[core]** **Verifiable rewards beat learned rewards wherever you can get them.** Your domain is unusually blessed here: GST schemas validate, forms have required fields, ledgers balance. Push as much of your reward signal as possible into programmatic verification and use an RM only for the residue.

- **[core]** Rubric-based and principle-conditioned reward models — judging against an explicit written criterion rather than opaque preference. More inspectable, more debuggable, and closer to how you’d defend a decision to a government auditor.
- **[skim]** Process reward models vs outcome reward models; AgentPRM.

Module 14 — Preference optimization

- Classic RLHF: PPO with an RM, KL penalty against the reference policy.
- **DPO** and derivatives: IPO, KTO, ORPO, SimPO. What each fixes.
- **When to use at all:** only when quality is judgeable but not programmatically verifiable. If you can write a checker, skip to Module 15.

Primary source for Modules 12–15: Nathan Lambert’s RLHF Book, free at rlhfbook.com, with a full lecture course at rlhfbook.com/course.

Module 15 — RL and RLVR

- **[core]** Policy gradient foundations: REINFORCE, baselines, advantage estimation, GAE.
- **[core]** PPO: clipping; why a critic is expensive.
- **[core]** **GRPO:** critic-free, group-normalized advantage, halves memory vs PPO. Failure modes — entropy collapse, advantage collapse, KL drift — are what you’ll actually spend time on.
- **[core]** **DAPO’s four fixes:** Clip-Higher (prevents entropy collapse), Dynamic Sampling, token-level policy gradient loss (essential for long CoT), overlong reward shaping. 50 AIME points on Qwen2.5-32B vs 47 for DeepSeek-R1-Zero-Qwen-32B, at 50% fewer steps.
- **[core]** **Dr. GRPO** (length-bias correction), **GSPO** (sequence-level ratios), **CISPO**.
- **[core]** Reward design and reward hacking. Budget more time here than on the algorithm.
- **[core]** Async RL: synchronous breaks at scale for long-horizon tasks. Rollout/trainer separation, stale-policy step semantics. Distributed systems problem — your strength.

Module 16 — Agentic RL and tool use as a training target

v1 treated tool use as something you evaluate. It's something you *train*.

- **[core]** Multi-turn credit assignment.
- **[core]** **Environment design — the real moat.** Verifiable outcomes, dense milestone rewards, sandboxing, determinism, reset semantics.
- **[core]** **Tool schema design as a training decision.** How tools are described in context, whether definitions are in the system prompt or a dedicated block, special tokens for call boundaries, parallel vs sequential calls. Models are sensitive to this, and a schema change at inference time silently invalidates training.
- **[core]** **Training error recovery explicitly.** A tool call fails, returns malformed JSON, or times out. Models that never saw failure in training loop forever on it. Inject failures into your environments deliberately — this is one of the highest-leverage, least-done things in agentic training.
- **[core]** Trajectory-level vs step-level advantages; GiGPO.
- **[core]** The compounding-error math: 95% per-step $\rightarrow$ 60% at 10 steps $\rightarrow$ 28% at 25. Everything in this module attacks that curve.
- **Frameworks in order:** OpenPipe ART $\rightarrow$ verl / verl-agent $\rightarrow$ SkyRL + SkyRL-Gym $\rightarrow$ NeMo Gym or OpenEnv for environment authoring.

Module 17 — Distillation

- **[core]** Off-policy vs **on-policy**. Compounding error goes from $O(\epsilon T^2)$ to $O(\epsilon T)$ — quadratic to linear in trajectory length. Decisive for long-horizon agentic work.
- **[core]** Forward KL (mode-covering $\rightarrow$ mass on low-probability teacher regions $\rightarrow$ hallucination) vs reverse KL (mode-seeking).
- **[core]** **The curse of capacity gap.** Larger teacher $\neq$ better student; there's a u-shaped optimum and the optimal teacher scale grows roughly linearly with student scale. Larger teachers produce less-soft logits and supervision that exceeds student capacity.
- **[core]** The “pseudo reasoning path” failure: SFT on expert traces produces output that looks like reasoning but contains redundant, hesitant, low-information, sometimes incorrect steps.

Reading: *A Survey of On-Policy Distillation for LLMs* (2604.00626) • *Distillation Scaling Laws* (2502.08606) • *Towards the Law of Capacity Gap* (2311.07052).

PART VI — BEHAVIOR (*entirely new*)

This part is the largest gap in v1, and it's most of what people mean when they say a model "feels" better than its benchmarks. It's also non-negotiable for government deployment.

Module 18 — Character and behavioral training

- **[core] A written specification of intended behavior comes first.** Anthropic's approach uses a constitution — a document addressed to the model, used at multiple stages of training to shape its character, with an explicit priority ordering (safe, ethical, compliant with guidelines, helpful). The core insight for you: values that are *explicitly stated in a document* can be inspected, argued about, and revised; values implicitly determined by preference data cannot. For a government-facing model, an inspectable behavioral spec is a compliance asset.
- **[core] Reason-based over rule-based.** The 2026 revision of Claude's spec shifted from giving rules to explaining the reasoning behind them, so the model can generalize to novel situations rather than pattern-match a prohibited list. Directly relevant: you cannot enumerate every situation a back-office agent will face.
- **[core] Consistency of persona across contexts and long conversations.** Character drift over a 25-turn agentic rollout is a real failure mode with no standard benchmark.
- **[core] Tone and register calibration** — and in your case, *per-language*. A model that is appropriately formal in English and inappropriately casual in Hindi has failed, and no standard eval will catch it.
- **[skim] Open Character Training (2511.01689)** for a public reproduction of the method.

Module 19 — Safety, refusal calibration, and red-teaming

- **[core] The helpfulness-harmlessness frontier.** The naive fix for unsafe behavior is more refusals, which produces a model nobody wants to use. CAI's claimed result was a Pareto improvement on both axes — the point is that treating this as a tradeoff is a failure of method, not a law of nature.

- **[core] Over-refusal is a real failure mode and needs its own eval set.** For a government tool, a model that refuses legitimate queries about tax law because they mention money is worse than useless.
- **[core] Red-teaming as a program:** adversarial datasets, jailbreak taxonomies, automated attack generation, tracking robustness across model versions.
- **[core] Prompt injection** — critical for you specifically. An agent that reads documents and calls tools will eventually read a document containing instructions. Train the distinction between data and instructions explicitly; it does not emerge for free.
- **[core] PII handling and data-residency behavior.** For Indian government workloads, what the model does with an Aadhaar number it encounters mid-workflow is a training-time question, not a prompt-time one.
- **[core] System cards and documented evaluation as a shipping artifact.** This is procurement paperwork you will need anyway; building it into the process is cheaper than retrofitting it.

Module 20 — Inference-time behavior as a training target

- **[core] Reasoning budget control.** Fused fast/slow thinking — one model that answers directly on easy prompts and deliberates on hard ones, with configurable effort levels — is a *trained behavior*, not an inference flag. Hy3 and others expose it as `reasoning_effort`. Given that on-prem economics make tokens free to your customer, this is your central lever, so train it deliberately.
 - **[core] Calibrated uncertainty and knowing when to abstain.** For financial audit and government filing, “I don’t have enough information to fill this field” is the correct and most valuable output. It has to be trained; base models will confabulate instead.
 - **[core] Instruction hierarchy** — system prompt vs user message vs tool output, and which wins. This is both a safety property and a product property.
 - **[skim] Length and verbosity control as an explicit objective.**
-

PART VII — MEASUREMENT AND SHIPPING

Module 21 — Evaluation (*promoted — build this first*)

You should build most of this **before** Part V, not after.

- **[core]** Contamination detection and eval hygiene.
- **[core]** **Benchmark validity.** A 2026 audit of BFCL v4, τ^2 -Bench, LiveMCPBench and MCP-Atlas found 92 evaluator–human disagreements across 496 expert-reviewed tasks — 18.5% misalignment — and rerunning LiveMCPBench with its default pipeline swung scores nearly 20 points *without changing the model*.
- **[core]** Variance across seeds; Pass@k vs Pass@1.
- **[core]** **Eval as CI.** Every change runs the full suite automatically; you never ship a regression you didn’t see. This is engineering discipline, not research, and it’s what makes a program compounding rather than random-walking.
- **[core]** **Behavioral evals, not just capability evals.** Refusal calibration, persona consistency, instruction hierarchy, tone per language. Nobody publishes these for Indic; you’ll build them.
- **[core]** Human preference evaluation done properly — blind, randomized, multiple raters, reported with confidence intervals.
- **[skim]** LLM-as-judge and its biases (position, verbosity, self-preference).

Module 22 — Model merging and checkpoint selection (*NEW*)

Cheap, underrated, and completely absent from v1.

- **[core]** Model soups — averaging checkpoints across a run or across runs. Often a free point or two.
- **[core]** Task arithmetic, TIES-merging, DARE. Merging a domain-specialized model back with the general model to recover lost general capability is a standard remedy for the over-SFT problem in Module 12.
- **[core]** **Which checkpoint do you actually ship?** Best-on-eval is usually wrong (you’ve overfit your eval through selection). Use a held-out selection set distinct from your reporting set.

Module 23 — Quantization in the training loop

[you mostly have this] — the new part is the training side.

- QAT mechanics: fake quantization in forward/backward, straight-through estimator.
- LLM-QAT (data-free, distillation-based), BitDistiller, EfficientQAT, OneBit/BinaryMoS.
- **Decision rule:** PTQ first, always. QAT only when PTQ misses your accuracy bar at your target bit-width — generally INT4 and below.
- Open question worth testing yourself: one comprehensive evaluation found little improvement from MXFP4 and NVFP4 over alternatives.

Module 24 — Instrumentation, debugging, and the gotchas

What to log

Loss (train + held-out), **gradient norm**, parameter norm, LR, tokens/sec, MFU, per-layer activation statistics, attention logit maxima, and for MoE: expert load distribution and router entropy. For RL, add: reward mean/variance, KL from reference, policy entropy, rollout length distribution, and a **random sample of raw rollouts**.

Gradient norm is a leading indicator — it typically rises before the loss spike appears.

The uncomfortable 2026 finding

Loss, gradient norms and weight norms are the *most delayed* indicators. After a numerical or hyperparameter fault destabilizes training, the run can continue thousands of steps looking normal while the fault is already written into weights and optimizer state. Mechanism-driven monitors — spectral entropy of a QK bilinear decomposition for low-precision attention, router-role indicators for MoE — fire thousands of steps before loss divergence. (2606.28116.)

Loss spikes

Usually aggressive LR schedules plus first/second-moment estimator lag in Adam-style optimizers. Mitigations in order of desperation: gradient clipping, longer warmup, lower peak LR, spike-aware optimizers with momentum reset, and the PaLM approach — auto-

restart from a recent checkpoint and skip the offending batch. Automate detect-and-restart before you need it.

The gotcha list

1. **Loss computed over prompt tokens** in SFT. Silent. Degrades everything.
2. **Chat template mismatch** between training and inference.
3. **Padding side** — left for generation, right for training.
4. **Special token ID drift** after tokenizer extension: untied embeddings, wrong EOS, generation never terminating.
5. **Packing without cross-document attention masks**.
6. **Broken resume** — optimizer state, RNG state, and dataloader position all must be restored, not just weights.
7. **Eval contamination** discovered after you've reported numbers.
8. **Copying a peak LR** from a paper at a different scale or batch size.
9. **Reward hacking that looks like success**. Reward climbs, model degrades, no exception thrown. Read raw rollouts weekly.
10. **Entropy collapse in GRPO** — exploration dies, diversity dies, reward curve stays flat-to-rising.
11. **Different generation parameters** at eval vs training time.
12. **Non-determinism you didn't intend** — seeds, cuDNN flags, dataloader worker order.
13. **MoE load imbalance** silently routing most tokens to a few experts.
14. **Over-SFT** past the OOD threshold — unrecoverable, invisible on training loss.
15. **Reward model drift** — the RM was valid for the initial policy and stopped being valid three hundred steps into RL. *(new)*
16. **Selecting the shipped checkpoint on your reporting eval** — you've overfit through selection and your published number is inflated. *(new)*
17. **Tool schema changed between training and deployment**. Silent, catastrophic, common. *(new)*
18. **Over-refusal introduced by safety training** and never measured because you only built harmfulness evals, not over-refusal evals. *(new)*

What you cannot copy from a frontier lab

Worth being honest about, so you optimize for the right things:

- **Ablation compute.** The real advantage isn't the final training run, it's having run two hundred careful experiments before it. You can partially substitute with sharper experimental design and smaller proxies — Module 0 is where you buy that back.
- **Years of accumulated eval infrastructure.** Every frontier lab has internal evals built over years that catch regressions no public benchmark sees. You start at zero here, which is exactly why Module 21 moves to the front.
- **Human data operations at scale.** Standing annotation programs with trained annotators and mature quality control. Your counter is domain focus: you need a hundred excellent GST-expert annotations, not a million generic ones.
- **Institutional taste.** People whose job is reading thousands of model outputs and noticing what's subtly wrong. Nothing substitutes; you build it by reading outputs yourself, weekly, forever. Start now.

What you have that they don't: a specific, narrow, high-value domain nobody has built environments for; unconstrained inference budget at deployment; and no legacy stack. That's a real hand. Play the narrow domain, not the general capability race.

Revised phase plan

Phase 0 — free, ~1 week. Module 0. Build the experiment tracker, change log, and scaling ladder. Do not skip this to get to the fun part.

Phase 1 — free to cheap, ~3 weeks. Modules 1, 2, 3, 21, 24. Write a 50M training loop from scratch. Deliberately induce a loss spike and diagnose it. Build the eval harness and the logging dashboard. Read the RLHF book end to end.

Phase 2 — one rented GPU, ~2–3 weeks. Modules 6, 12, 15. Start with rejection sampling — it's cheaper than RL and often most of the gain. Then implement GRPO yourself on GSM8K with ART/Unsloth; verifiable math is the cheapest RL sandbox that exists. Reproduce a published number so you know what healthy curves look like.

Phase 3 — multi-GPU. Modules 7, 13, 16, 17. verl or SkyRL, multi-turn tool use, your own environments, on-policy distillation.

Phase 4 — before anything ships. Modules 18, 19, 20, 22. Behavioral spec, safety and over-refusal evals, reasoning-budget training, checkpoint selection. Government procurement will ask for the artifacts these produce, and retrofitting them costs more than building them in.

The rule: never rent a large cluster to learn what a small one can teach you. Every hour of Phase 0–1 saves roughly a day of Phase 3.